Лабораторная работа № 16

Тема: Обработка асинхронных запросов с использованием async/await и обработка ошибок.

Цель: Научиться использовать синтаксис async/await для выполнения HTTP-запросов, корректно обрабатывать сетевые ошибки и ошибки статуса сервера, а также работать с динамическими данными из публичных API.

Вариант 1

Задание 1. Получение профиля пользователя (JSONPlaceholder)
Используя async/await, получите данные пользователя с id=3 по адресу https://jsonplaceholder.typicode.com/users/3. Выведите в консоль имя пользователя, его email и название компании (company.name).

Задание 2. Список дел (JSONPlaceholder)
Напишите асинхронную функцию, которая загружает список задач (/todos) пользователя с id=1. Отфильтруйте массив и выведите в консоль только названия (title) тех задач, которые не завершены (completed: false).

Задание 3. Случайные факты о котах (Cat Facts API)
Сделайте запрос к https://catfact.ninja/fact. Используя async/await, получите случайный факт и выведите его в консоль. Добавьте проверку: если длина факта больше 100 символов, выведите сообщение «Длинный факт», иначе — «Короткий факт».

Задание 4. Информация о конкретной стране (REST Countries)
Получите данные о стране «Германия» (https://restcountries.com/v3.1/name/germany). Выведите столицу страны и общее количество населения.

Задание 5. Комплексная обработка ошибок
Напишите функцию, которая делает запрос к https://jsonplaceholder.typicode.com/posts/999 (несуществующий пост).
Используйте try...catch.
Внутри try проверьте response.ok. Если оно false, выбросьте ошибку через throw new Error.
В блоке catch выведите сообщение: «Запрос не удался: [текст ошибки]».

Вариант 2
Задание 1. Получение поста и комментариев (JSONPlaceholder)

Используя async/await, получите данные поста с id=5 (/posts/5). Выведите в консоль заголовок поста. Затем сделайте второй запрос, чтобы получить все комментарии к этому посту (/posts/5/comments), и выведите их количество.

Задание 2. Фильтрация альбомов (JSONPlaceholder)
Напишите асинхронную функцию, которая получает список всех фотоальбомов (/albums). Выведите в консоль заголовки (title) первых 5 альбомов из полученного списка.

Задание 3. Университеты страны (Hipolabs University API)
Сделайте запрос к http://universities.hipolabs.com/search?country=Kazakhstan. Выведите названия первых трех университетов из списка. Если список пуст, выведите «Данные не найдены».

Задание 4. Случайный пользователь (Random User API)
Используя https://randomuser.me/api/, получите данные одного случайного пользователя. Выведите в консоль его полное имя (first + last) и город проживания.

Задание 5. Имитация сетевого сбоя и ошибки 404
Создайте функцию, которая принимает URL.
Вызовите её с некорректным доменом (например, https://wrong-domain-123.com), чтобы спровоцировать сетевую ошибку.
Вызовите её с адресом https://jsonplaceholder.typicode.com/invalid.
В блоке catch реализуйте разную обработку: если это ошибка статуса (например, 404), пишите «Ресурс не найден», если сетевая ошибка — «Проверьте интернет-соединение».

Шпаргалка: fetch, async/await и статус ответа
1. Простейший запрос с выводом статуса

```
async function demo() {
  let response = await
fetch("https://jsonplaceholder.typicode.com/todos/1");
  console.log("Статус ответа:", response.status);

  let data = await response.json();
  console.log("Данные:", data);
}

demo();
```

📌 Здесь видно:
response.status → числовой код (200, 404 и т. д.),
данные преобразуются в JSON.

2. Проверка статуса через ok

```
async function checkStatus() {
  let response = await
fetch("https://jsonplaceholder.typicode.com/unknown");
  console.log("Статус ответа:", response.status);

  if (!response.ok) {
    console.error("Ошибка HTTP:", response.status);
    return;
  }

  let data = await response.json();
  console.log("Данные:", data);
}

checkStatus();
```

📌 Если сервер вернёт 404, мы это заметим через ok.

3. Сетевая ошибка (try/catch)

```
async function networkFail() {
  try {
    let response = await fetch("https://wrong-
domain.example.com/data");
    console.log("Статус ответа:", response.status); // сюда
не дойдёт
    let data = await response.json();
    console.log("Данные:", data);
  } catch (error) {
    console.error("Сетевая ошибка:", error.message);
  }
}

networkFail();
```

📌 Ошибки сети (например, нет интернета) ловятся только через catch.

4. Работа с текстом, а не JSON

```
async function getText() {
  let response = await fetch("https://httpbin.org/html");
  console.log("Статус ответа:", response.status);

  let text = await response.text();
  console.log("HTML длиной:", text.length, "символов");
}

getText();
```

📌 У объекта Response можно вызывать не только .json(), но и .text(), .blob(), .arrayBuffer().

5. Универсальная функция для запросов

```
async function fetchData(url) {
  try {
    let response = await fetch(url);
    console.log("Статус:", response.status);

    if (!response.ok) {
      throw new Error("Ошибка HTTP " + response.status);
    }

    let data = await response.json();
    return data;
  } catch (error) {
    console.error("Ошибка запроса:", error.message);
  }
}
```

// Пример использования:

```
fetchData("https://jsonplaceholder.typicode.com/todos/2")
  .then(data => console.log("Задача:", data));
```

📌 Такая функция помогает не дублировать проверку статуса в каждом запросе.

⚡ **Мини-таблица полезных статусов HTTP**

200 — OK (успешно).

201 — Created (создано, обычно для POST).
400 — Bad Request (ошибка клиента).
401 — Unauthorized (нужна авторизация).
403 — Forbidden (доступ запрещён).
404 — Not Found (ресурс не найден).
500 — Internal Server Error (ошибка сервера).

Используйте response.status для проверки кода.
response.ok помогает быстро понять, успешный ли запрос.
Сетевые ошибки ловятся только в try/catch.
Помимо .json() есть и другие методы для чтения ответа.
Универсальная функция fetchData экономит время и уменьшает ошибки.